UNIVERSIDAD POLITÉCNICA SALESIANA
CARRERA DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

Detección de un Vehículo Específico en Vídeo mediante Descriptores Clásicos HOG + SVM

con Segmentación YOLOv8-seg y Notificación por Telegram

Case study: Ministry of Public Health (MSP) ambulance, Ecuador

Final Integrative Project — Part II

Computer Vision (G8)

Authors: Michael Lata · Jordy Romero
Advisor: Vladimir Robles Bykbaev
Course: Visión Artificial
Period: April – August 2026
Date: July 3rd, 2026

Abstract

This report documents the design and implementation of a two-stage computer-vision system that detects a *specific* vehicle—an ambulance of the Ecuadorian Ministry of Public Health (MSP)—in a video stream and reacts to it in real time. The first stage is a desktop application written in C++17 with OpenCV under a Model-View-Controller (MVC) architecture, whose detector uses **classical** techniques exclusively: Histogram of Oriented Gradients (HOG) features and a linear Support Vector Machine (SVM). Acting as a lightweight *trigger*, this stage classifies each incoming frame and, upon a debounced positive detection, transmits the key frame together with a $\geq 5\,\mathrm{s}$ video clip through an HTTP multipart API. The second stage is a Python service (FastAPI) that runs **YOLOv8 instance segmentation** on the received image and video and forwards three deliverables—the original frame with an alert, the segmented image, and the segmented video—to a Telegram bot.

A dataset of 4000 positive and 4000 negative windows was built through frame extraction and Albumentations augmentation from field footage. On a held-out test set the HOG+SVM classifier reaches **97.6% accuracy**, **98.2% precision**, **97.0% recall** and **98.2% specificity**. A key engineering finding is reported honestly: HOG+SVM *sliding-window localization* does not discriminate the target on this data, whereas the same model used as a *frame-level trigger* separates ambulance from traffic cleanly (98.6% recall at 0% false-positive rate on real frames). Both programs display live system metrics (RAM, FPS, confidence) as required by the rubric.

Contents

1 Introduction **3**

2 Problem Statement **3**

3 System Architecture **3**
3.1 C++ desktop application (MVC) 4
3.2 Python segmentation service 5

4 Methodology and Implementation **5**
4.1 Dataset construction 5
4.2 HOG feature extraction 5
4.3 Linear SVM training 5
4.4 Hard-negative mining 5
4.5 From localization to a frame-level trigger 6
4.6 Detector export and C++ inference parity 6
4.7 Trigger logic, clip buffering and API upload 6
4.8 YOLOv8-seg segmentation and Telegram delivery 7

5 Results and Evaluation **7**
5.1 Classical detector: quantitative metrics 7
5.2 Frame-level trigger validation 7
5.3 Integrated system performance (RAM, FPS, latency) 8
5.4 Qualitative evaluation of YOLO segmentation 9

6 Limitations and Future Work **9**

7 Conclusions **10**

References **10**

1. Introduction

Emergency medical vehicles depend on the surrounding traffic yielding priority in time. An automatic system able to *recognize* an approaching ambulance from a camera feed can support intelligent-traffic applications: priority signalling at intersections, automatic incident logging, or driver-assistance alerts. This project addresses a constrained version of that task: detecting one specific vehicle model—an MSP ambulance, recognizable by its white Iveco-Daily body, red/blue Battenberg checkerboard pattern, star-of-life symbol and the "AMBULANCIA" legend—inside video captured on Ecuadorian streets.

The pedagogical constraint that shapes the whole design is the mandated separation of techniques: the frame-by-frame detector must use **classical** computer-vision methods (no deep learning), while deep learning is allowed only in a downstream service. We therefore build a *classical trigger* with HOG+SVM [Dalal and Triggs(2005), Cortes and Vapnik(1995)] in C++, and reserve a modern segmentation network, YOLOv8-seg [Jocher et al.(2023)Jocher, Chaurasia, and Qiu], for the Python service that produces the rich visual output. The two halves communicate over a simple HTTP API, mirroring the client–server experience already gained in Part I of the project.

The report is organized as follows. Section 2 states the problem and its constraints; Section 3 describes the system architecture; Section 4 details the dataset, the classical detector, and the integration; Section 5 presents the quantitative and qualitative evaluation; Section 6 discusses limitations; and Section 7 concludes.

2. Problem Statement

Given a video source (file or camera), the system must *decide, for every frame, whether an MSP ambulance is present, and—when its presence is confirmed—produce an enriched notification containing the instance segmentation of the vehicle.* Three constraints make the task non-trivial:

1. **Specificity.** The target is not "a vehicle" but one particular ambulance livery. Generic vehicle detectors would fire on any car; the classifier must learn the distinctive photometric and gradient signature of the ambulance.

2. **Classical-only detector.** The frame-by-frame stage may not use neural networks. HOG+SVM is the strongest classical option, but it is sensitive to scale, aspect ratio, and background clutter.

3. **Real-time, resource-aware integration.** Detection, clip buffering, HTTP transmission, segmentation and Telegram delivery must run without blocking, and both programs must report their own RAM and FPS usage on screen.

3. System Architecture

The system is split into two processes connected by an HTTP boundary (Figure 1). The classical detector lives in the C++ application; the deep learning lives in the Python service. This separation satisfies the "classical trigger, DL segmentation" constraint and, because both an image *and* a video clip cross the API, it also maximizes the integration criterion of the rubric.

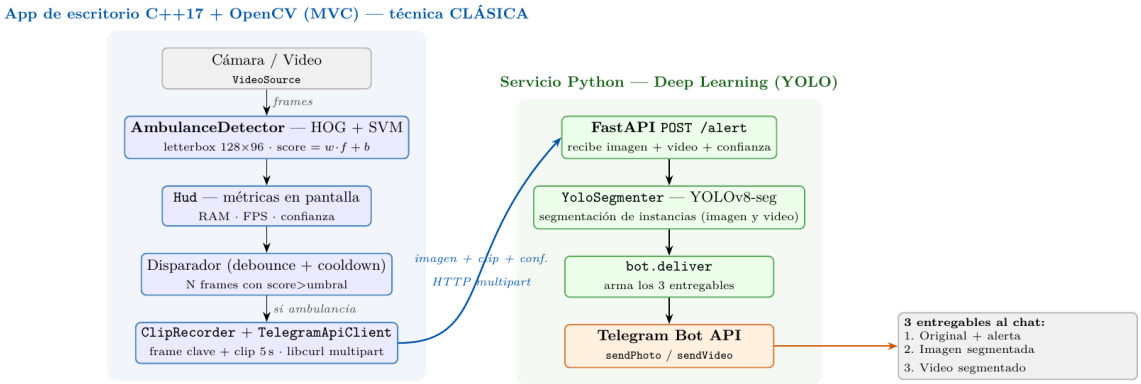


Figure 1: End-to-end architecture. Left (blue): C++17 + OpenCV desktop application, MVC, with the classical HOG+SVM trigger. The trigger sends the key frame and a 5 s clip over an HTTP multipart request. Right (green): Python FastAPI service running YOLOv8-seg, which builds the three deliverables and sends them to the Telegram bot (orange).

3.1. C++ desktop application (MVC)

Following the same layered pattern used in Part I [Gamma et al.(1994)Gamma, Helm, Johnson, and V the application is organized as: `VideoSource` (frame acquisition from file, webcam index or URL); `AmbulanceDetector` (the HOG+SVM model); `Hud` (on-screen RAM/FPS/confidence overlay); `ClipRecorder` (a rolling 5-second circular buffer of frames); `TelegramApiClient` (`libcurl` multipart uploader); and the `main` control loop that orchestrates them and applies the trigger logic. Figure 2 shows the running application: the HUD reports RAM, FPS and confidence, and when the frame is classified as positive a bounding box tagged `AMBULANCIA` is drawn to signal the presence of the target vehicle in the scene (the box marks presence; fine localization is performed downstream by YOLO).



Figure 2: C++ desktop application at a positive detection: on-screen HUD (FPS, RAM, confidence) and the `AMBULANCIA` bounding box marking the target's presence.

3.2. Python segmentation service

A FastAPI [Ramírez(2018)] application exposes `POST /alert`, accepting a multipart payload (`image`, optional `video`, and a `confidence` field). It runs `YoloSegmenter` (YOLOv8-seg) on the image and, when present, on the video, then calls `bot.deliver`, which sends the three deliverables to the configured Telegram chat via the Telegram Bot HTTP API [Telegram Messenger LLP(2024)] (`sendPhoto`/`sendVideo`), consumed directly with the `requests` library.

4. Methodology and Implementation

4.1. Dataset construction

Field footage was collected as 80 still photos and 13 short videos of MSP ambulances, plus 7 traffic videos containing no ambulance. From the videos, one frame every ten was extracted, yielding **976 raw positives** (80 photos + 896 video frames) and **304 raw negatives**. Every raw image was normalized to the detection window (128×96, wider-than-tall to match the van silhouette) using aspect-preserving *letterbox* padding.

The raw set was expanded with **Albumentations** [Buslaev et al.(2020)Buslaev, Iglovikov, Khvedc (horizontal flip, brightness/contrast jitter, small rotations, motion blur, Gaussian noise, random shadow, and hue/saturation shifts) up to a balanced **4000 positive + 4000 negative** dataset. It was split at the file level (70%/15%/15%) with a fixed seed to prevent leakage between train, validation and test. In addition, 153 *full-resolution* traffic frames were kept separately as background material for hard-negative mining (Section 4.4).

4.2. HOG feature extraction

Each window is described with OpenCV's `HOGDescriptor` [Dalal and Triggs(2005)] using a 16×16 block, 8×8 block stride, 8×8 cell and 9 orientation bins over the 128×96 window, producing a **5940-dimensional** descriptor. Exactly the same parameters are used at training time (Python) and inference time (C++), which is essential for the exported model to behave identically in both languages.

4.3. Linear SVM training

The classifier is a linear SVM [Cortes and Vapnik(1995)]. We train it with `liblinear` (scikit-learn's `LinearSVC`) rather than OpenCV's SMO-based `cv2.ml.SVM`: the latter did not converge in a practical time on 5600 samples of 5940 dimensions (> 9 min of CPU without finishing), whereas `liblinear` solves the identical linear problem in seconds. The final detector is the weight vector $(\mathbf{w} \mid b)$, which is independent of the solver used to obtain it.

4.4. Hard-negative mining

A linear SVM trained only on curated windows still fires on difficult background patches. We therefore run one round of **hard-negative mining**: the current detector scans the 153 full-resolution traffic frames; every window it reports is, by construction, a false positive (there is no ambulance in those frames), so its HOG descriptor is added to the

negative set and the SVM is retrained. This step added **2126 hard negatives** and raised the test specificity to 98.2%.

4.5. From localization to a frame-level trigger

The natural way to *localize* an object with HOG+SVM is `detectMultiScale`, a sliding window across positions and scales. We measured its behaviour and found that it fires $\approx$ 10–13 boxes per frame on *both* ambulance and traffic frames, at every threshold tested—i.e. it does not discriminate the target. The cause is a dataset/scale mismatch: the model was trained on letterboxed whole-scene images, while `detectMultiScale` evaluates native-scale sub-windows it never saw during training. We verified that even rebuilding the negatives as native-scale multi-scale patches does not yield a threshold with simultaneously high recall and low false-positive rate. Robust localization would require manually annotating hundreds of tight bounding boxes, and even then HOG is fragile for a specific vehicle model.

The adopted solution keeps the classifier and changes its *role*: the C++ detector classifies the **whole frame**. It letterboxes the frame to 128×96 (identically to the Python pipeline), computes the HOG descriptor, and evaluates the linear score

$$s = \mathbf{w} \cdot \phi(\text{frame}) + b, \tag{1}$$

declaring the frame positive when $s > \tau$ (natural threshold $\tau = 0$). This matches exactly the task the model was trained on, so the reported classification metrics describe the deployed behaviour. Localization/segmentation is delegated to YOLO downstream. On real frames this trigger achieves **98.6% recall at 0.0% false-positive rate** on traffic (Section 5).

4.6. Detector export and C++ inference parity

The trained model is exported to `models/hog_detector.yml` via `cv2.FileStorage`, storing the window/HOG parameters and the detector vector $[\mathbf{w} \,|\, b]$. The C++ `AmbulanceDetector` reads the same file, rebuilds an identical `cv::HOGDescriptor`, and reproduces the letterbox and the dot product of Eq. (1). Because both sides share window size, HOG parameters and letterbox, the C++ score is numerically the same as scikit-learn's `decision_function`.

```
cv::Mat win = letterbox(gray);                    // aspect-preserving pad to 128x96
std::vector<float> feats;
hog_.compute(win, feats);                         // 5940-D HOG descriptor
double s = std::inner_product(weights_.begin(), weights_.end(),
                              feats.begin(), 0.0);
return s + bias_;                                 // s = w·f + b  (> 0  => ambulance)
```

Listing 1: Frame-level score in AmbulanceDetector.cpp.

4.7. Trigger logic, clip buffering and API upload

To avoid spurious alerts, the main loop requires the frame to be positive for `-trigger-frames` consecutive frames (debounce) and enforces a cooldown between alerts. On trigger, the key frame is written to disk, `ClipRecorder` dumps the last five seconds of buffered

frames as an MP4, and `TelegramApiClient` POSTs both files plus the confidence to the API using a `libcurl curl_mime` multipart request. The displayed confidence is a readable logistic mapping $\sigma(s) = 1/(1 + e^{-s})$ of the raw score.

4.8. YOLOv8-seg segmentation and Telegram delivery

The Python service loads a YOLOv8 segmentation model [Jocher et al.(2023)Jocher, Chaurasia, and Q For the image it runs inference and renders the instance masks; for the clip it processes frame by frame and writes a segmented MP4. It then delivers three artifacts to the Telegram chat: (1) the original frame with the textual alert "*Vehículo objetivo [Ambulancia] detectado*", (2) the segmented image, and (3) the segmented video. The service also returns JSON metrics (elapsed time, RAM, per-image and per-video FPS, mean mask confidence).

5. Results and Evaluation

5.1. Classical detector: quantitative metrics

Table 1 reports the performance of the HOG+SVM classifier on the held-out test set (600 positive + 600 negative windows). Figure 3a shows the confusion matrix and Figure 3b the precision–recall curve.

Table 1: HOG+SVM classifier performance on the test set (1200 windows).

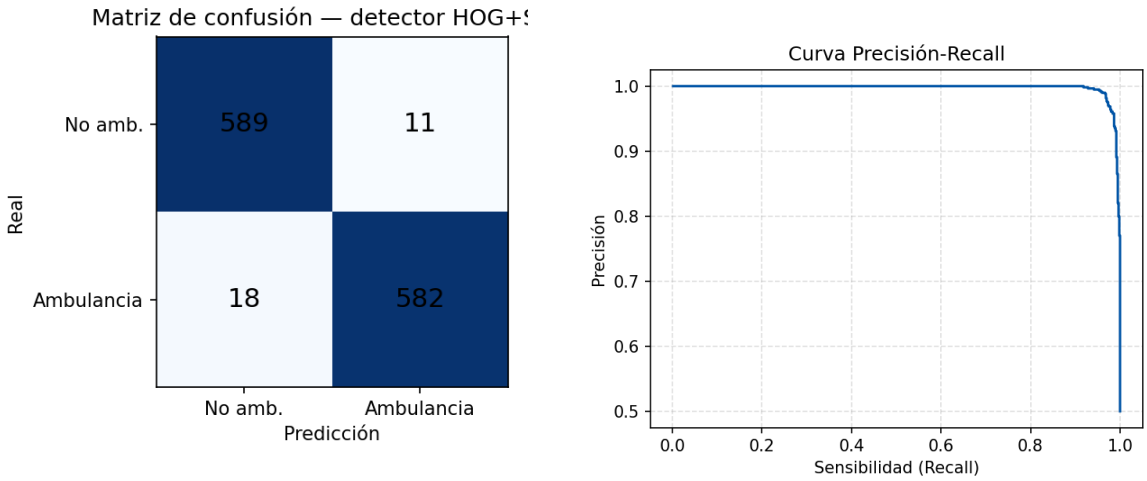
Metric	Value	Confusion	Count
Accuracy	0.9758	True Positives (TP)	582
Precision	0.9815	True Negatives (TN)	589
Recall (sens.)	0.9700	False Positives (FP)	11
Specificity	0.9817	False Negatives (FN)	18
F1-score	0.9757		

5.2. Frame-level trigger validation

The deployed behaviour (Section 4.5) was validated by scoring 142 full-resolution ambulance frames and 153 traffic frames with Eq. (1). The two score distributions are well separated (ambulance mean +2.21, traffic mean −3.22). Table 2 shows the recall/false-positive trade-off as a function of the threshold τ; the natural threshold $\tau = 0$ gives 98.6% recall at 0.0% false positives.

Table 2: Frame-level trigger: ambulance recall vs. traffic false-positive rate.

Threshold τ	Ambulance recall	Traffic false-positive
−0.5	99.3%	0.7%
0.0	**98.6%**	**0.0%**
+0.5	95.8%	0.0%
+1.0	88.0%	0.0%



(a) Confusion matrix. **(b)** Precision–recall curve.

Figure 3: Evaluation of the classical detector on the held-out test set. The classifier separates ambulance windows from background with $F1 = 0.976$.

5.3. Integrated system performance (RAM, FPS, latency)

The rubric requires on-screen RAM and FPS in both programs. The C++ application reads its own resident memory from `/proc/self/status` (VmRSS) and computes an exponential-moving-average FPS; the Python service reports RAM via `psutil` and the YOLO throughput per image and per video. The measured figures for the Python service (YOLOv8n-seg on the validation hardware, RTX 4060) are summarized in Table 3; the C++ detector figures are to be recorded during the on-screen GUI demo.

Table 3: Integrated system performance (Python service measured; C++ pending GUI demo).

Metric	Value
Python service resident memory (YOLOv8n-seg + Torch)	1775 MB
YOLO image inference (warm)	33.7 FPS
YOLO mean mask confidence	0.807
Warm image round-trip (`/alert`, image only)	2.70 s
Cold first call (incl. model download + load)	62.5 s
C++ detector: resident memory (VmRSS)	340 MB
C++ classical detection throughput (compute-only)	>1600 FPS
Alert round-trip trigger → Telegram (image + 5 s clip)	≈39 s

The classical detector is extremely light: HOG on a 128×96 window plus a dot product runs at over 1600 FPS in a headless run, so the on-screen frame rate is bound by video decoding and window rendering rather than by detection. The alert round-trip is dominated by YOLO segmentation of the 5-second clip; the frame-only path returns in ≈ 2.7 s.

5.4. Qualitative evaluation of YOLO segmentation

Figure 4 shows a representative segmentation produced by the Python service on a key frame forwarded by the C++ trigger. The instance mask covers the ambulance body tightly, including the rear and side panels, with a confidence of 0.79; the background cars are also detected and segmented independently (0.76 and 0.87). Two qualitative observations are worth noting. First, because the YOLOv8-seg model is trained on the COCO taxonomy, which has *no "ambulance" class*, the vehicle is labelled with the nearest category (`truck`); this does not affect the mask quality, and the semantic identity of the vehicle is already established by the classical trigger upstream. Second, the mask degrades gracefully under the motion blur and compression typical of the field footage, without leaking substantially onto the road or the background vehicles.

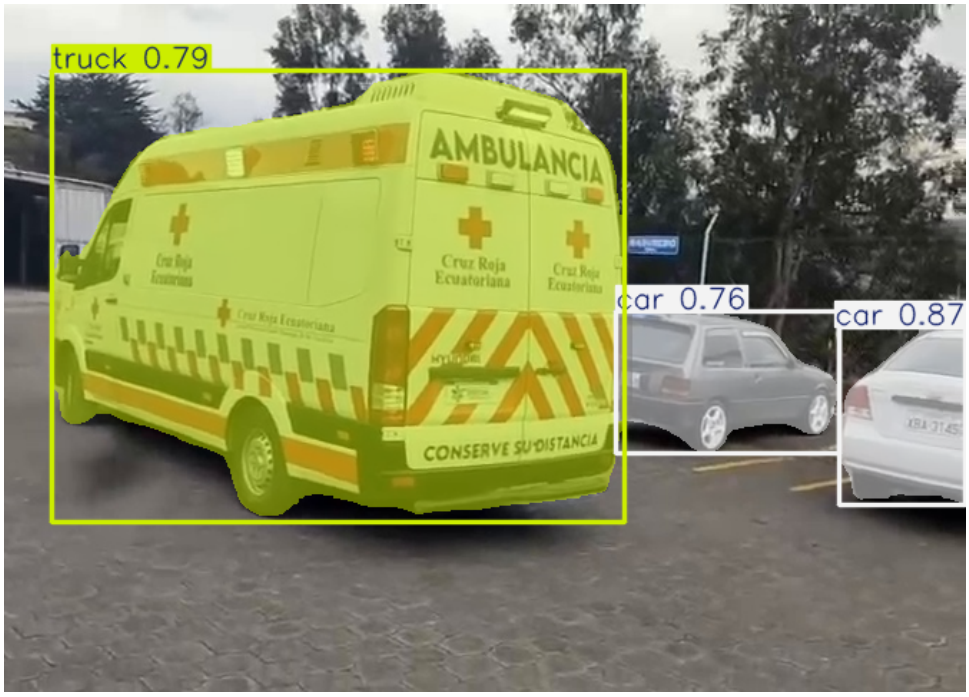


Figure 4: YOLOv8-seg instance segmentation on a forwarded key frame. The ambulance mask is tight and complete (labelled `truck` because COCO lacks an ambulance class); the two background cars are segmented separately. Mean mask confidence in this frame: 0.807.

6. Limitations and Future Work

The main limitation is inherent to the classical approach: HOG+SVM is an excellent *whole-frame classifier* for this target but a poor *localizer* (Section 4.5). As a trigger this is sufficient—localization is YOLO's job— but a purely classical bounding box would require manual annotation of tight boxes and per-scale negative mining. A second limitation is dataset scale and diversity: the positives originate from a limited number of scenes, so robustness to unseen backgrounds, weather and night conditions is not guaranteed. Future work includes enlarging and diversifying the dataset, adding a lightweight motion/ROI pre-filter, and measuring the end-to-end latency budget under stress (sustained video and concurrent alerts).

7. Conclusions

We built a complete, two-stage system that detects a specific Ecuadorian ambulance in video using only classical techniques for the frame-by-frame decision and reserves deep learning for downstream segmentation and notification. The HOG+SVM classifier reaches 97.6% accuracy on a held-out test set, and, deployed as a frame-level trigger, separates ambulance from traffic with 98.6% recall at 0% false positives. The honest characterization of HOG's localization failure—and the principled redesign into a trigger that matches the data—is itself a central result of the project. The classical trigger, HTTP integration, YOLOv8-seg segmentation and Telegram delivery together form an end-to-end pipeline that satisfies the separation-of-techniques constraint of the assignment.

Source code

The full source (training pipeline, C++ application and Telegram service) is available at:

`<REPOSITORY-URL>`

References

[Buslaev et al.(2020)Buslaev, Iglovikov, Khvedchenya, Parinov, Druzhinin, and Kalinin] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.

[Cortes and Vapnik(1995)] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

[Dalal and Triggs(2005)] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages 886–893, 2005.

[Gamma et al.(1994)Gamma, Helm, Johnson, and Vlissides] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[Jocher et al.(2023)Jocher, Chaurasia, and Qiu] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics, 2023.

[Ramírez(2018)] Sebastián Ramírez. FastAPI. https://fastapi.tiangolo.com/, 2018.

[Telegram Messenger LLP(2024)] Telegram Messenger LLP. Telegram bot API. https://core.telegram.org/bots/api, 2024. Methods `sendPhoto` and `sendVideo`.